

## Python Code

```
1  import streamlit as st
2  import pandas as pd
3  import numpy as np
4  import joblib
5  import matplotlib.pyplot as plt
6  import seaborn as sns
7
8  st.set_page_config(page_title="Bank Churn Dashboard", layout="wide")
9
10 # load data + model
11
12 #-----
13
14 @st.cache_data
15 def load_data():
16     df = pd.read_csv("European_Bank.csv")
17
18     # feature engineering (must match training )
19     df['EngagementScore'] = df['IsActiveMember'] * df['NumOfProducts']
20     df['BalanceSalaryRatio'] = df['Balance'] / (df['EstimatedSalary'] + 1)
21     df['TenureAgeRatio'] = df['Tenure'] / (df['Age'] + 1)
22
23     df['ProductUsageLevel'] = pd.cut(
24         df['NumOfProducts'],
25         bins=[0,1,2,4],
26         labels=['Low', 'Medium', 'High']
27     )
28
29     threshold = df['Balance'].quantile(0.75)
30     df['IsHighValueCustomer'] = (df['Balance'] > threshold).astype(int)
31
32
33     df['IsDisengagedHighValue'] = (
34         (df['IsHighValueCustomer'] == 1) &
35         (df['IsActiveMember'] == 0)
36     ).astype(int)
37
38     return df
39
40 @st.cache_resource
41 def load_model():
42     return joblib.load("churn_model.pkl")
43
44 df = load_data()
45 model = load_model()
46
47 #-----
48
```

```

49 # SIDEBAR FILTERS
50
51 st.sidebar.header("🔍 Filter Customers")
52
53 age_range = st.sidebar.slider("Age", int(df.Age.min()), int(df.Age.max()),
54                               (20,50))
55 balance_range = st.sidebar.slider("Balance", int(df.Balance.min()),
56                                   int(df.Balance.max()), (0,100000))
57 products_filter = st.sidebar.multiselect("NumOfProducts",
58                                           df.NumOfProducts.unique(), default=df.NumOfProducts.unique())
59 active_filter = st.sidebar.multiselect("IsActiveMember", [0,1], default=[0,1])
60
61 #-----
62
63 # Apply filters
64 filtered_df = df[
65     (df['Age'].between(age_range[0], age_range[1])) &
66     (df['Balance'].between(balance_range[0], balance_range[1])) &
67     (df['NumOfProducts'].isin(products_filter)) &
68     (df['IsActiveMember'].isin(active_filter))
69 ]
70
71 #-----
72
73 # KPIs
74 def calculate_kpis(data):
75
76     ERR = 1 - data[data['IsActiveMember']==1]['Exited'].mean()
77     PDI = data['NumOfProducts'].mean()
78     HBDR = data[data['IsHighValueCustomer']==1]
79     ['IsActiveMember'].value_counts(normalize=True).get(0,0)
80     CCS = data[data['HasCrCard']==1]['Exited'].mean()
81     RSI = data['EngagementScore'].mean()
82
83     return ERR, PDI, HBDR, CCS, RSI
84
85 ERR, PDI, HBDR, CCS, RSI = calculate_kpis(filtered_df)
86
87 #-----
88
89 # TITLE
90 st.title("🏠 Banking Customer Churn Dashboard")
91
92 #-----
93
94 # KPI DISPLAY
95 col1, col2, col3, col4, col5 = st.columns(5)
96
97 col1.metric("Engagement Retention", f"{ERR:.2f}")
98 col2.metric("Product Depth", f"{PDI:.2f}")

```

```
95 col3.metric("High Balance Risk", f"{HBDR:.2f}")
96 col4.metric("Credit Card Churn", f"{CCS:.2f}")
97 col5.metric("Relationship Strength", f"{RSI:.2f}")
98
99 #-----
100 -----
101 # SECTION 1: CHURN OVERVIEW
102 st.header("1 Churn Overview")
103
104 col1, col2 = st.columns(2)
105
106 with col1:
107     fig, ax = plt.subplots()
108     sns.countplot(x='Exited', data=filtered_df, ax=ax)
109     ax.set_title("Churn Distribution")
110     st.pyplot(fig)
111
112 with col2:
113     churn_rate = filtered_df['Exited'].mean()
114     st.write(f"### Churn Rate: {churn_rate:.2%}")
115
116 #-----
117 -----
118 # SECTION 2: ENGAGEMENT ANALYSIS
119 st.header("2 Engagement Analysis")
120
121 col1, col2 = st.columns(2)
122
123 with col1:
124     fig, ax = plt.subplots()
125     sns.barplot(x='IsActiveMember', y='Exited', data=filtered_df, ax=ax)
126     ax.set_title("Churn vs Activity")
127     st.pyplot(fig)
128
129 with col2:
130     fig, ax = plt.subplots()
131     sns.histplot(filtered_df['EngagementScore'], kde=True, ax=ax)
132     ax.set_title("Engagement Score Distribution")
133     st.pyplot(fig)
134
135 #-----
136 -----
137 # SECTION 3: PRODUCT UTILIZATION
138 st.header("3 Product Utilization")
139
140 col1, col2 = st.columns(2)
141
142 with col1:
143     fig, ax = plt.subplots()
144     sns.barplot(x='NumOfProducts', y='Exited', data=filtered_df, ax=ax)
145     ax.set_title("Churn vs Number of Products")
```

```

146         st.pyplot(fig)
147
148     with col2:
149         fig, ax = plt.subplots()
150         sns.countplot(x='ProductUsageLevel', data=filtered_df, ax=ax)
151         ax.set_title("Product Usage Segments")
152         st.pyplot(fig)
153
154     # SECTION 4: HIGH-RISK CUSTOMERS
155     st.header("🔍 High-Risk Customer Detector")
156
157     high_risk_df = filtered_df[
158         filtered_df['IsDisengagedHighValue'] == 1
159     ]
160
161     st.write(f"High-Risk Customers Found: {len(high_risk_df)}")
162
163     if high_risk_df.empty:
164         st.warning("No high-risk customers match current filters")
165     else:
166         st.dataframe(high_risk_df.head(20), use_container_width=True)
167
168     #-----
169
170     # MODEL PREDICTION
171     st.subheader("🧠 Predict Customer Churn")
172
173     col1, col2, col3 = st.columns(3)
174
175     with col1:
176         age = st.number_input("Age", 18, 80, 30)
177         balance = st.number_input("Balance", 0, 200000, 50000)
178
179     with col2:
180         products = st.selectbox("NumOfProducts", [1,2,3,4])
181         active = st.selectbox("IsActiveMember", [0,1])
182
183     with col3:
184         credit_score = st.number_input("CreditScore", 300, 900, 600)
185         salary = st.number_input("EstimatedSalary", 10000, 200000, 50000)
186
187     #-----
188
189     # Create input
190     input_data = pd.DataFrame({
191         'CreditScore': [credit_score],
192         'Geography': ['France'],
193         'Gender': ['Male'],
194         'Age': [age],
195         'Tenure': [5],
196         'Balance': [balance],
197         'NumOfProducts': [products],

```

```
198     'HasCrCard':[1],
199     'IsActiveMember':[active],
200     'EstimatedSalary':[salary],
201
202     # ✅ MATCH TRAINING FEATURES EXACTLY
203     'EngagementScore':[active * products],
204     'BalanceSalaryRatio':[balance / (salary + 1)],
205
206     # ⚠️ FIX NAME (was TenureAgeRatio)
207     'TenureRatio':[5 / (age + 1)],
208
209     'ProductUsageLevel':['Medium'],
210     'IsHighValueCustomer':[1 if balance > df['Balance'].quantile(0.75) else 0],
211     'IsDisengagedHighValue':[1 if (balance > df['Balance'].quantile(0.75) and
active==0) else 0],
212
213     # ⚠️ ADD MISSING COLUMN
214     'Year':[2024] # or whatever default used in training
215 })
216
217 if st.button("Predict"):
218     prob = model.predict_proba(input_data)[0][1]
219
220     st.write(f"### Churn Probability: {prob:.2f}")
221
222     if prob > 0.5:
223         st.error("⚠️ High Risk Customer")
224     else:
225         st.success("✅ Low Risk Customer")
```